SIH1641: Annual Report Portal

INSTITUTE DEPARTMENTAL REPORT INTEGRATION & CUSTOMIZATION PLATFORM

Academic Project Log (Phase 1): June 01 – June 08, 2025

Domain: Institutional ERP / Enterprise Information Systems

Tech Stack: React.js / Next.js | Node.js (Express) | PostgreSQL or MySQL

Activity Log Summary

Day	Date	Activity	Status
Day 1	June 01, 2025	Project Scope & Title Finalization	✔ Completed
Day 2	June 02, 2025	System Requirement Gathering	✔ Completed
Day 3	June 03, 2025	Objective & Boundary Definition	✔ Completed
Day 4	June 04, 2025	User Roles & Module Identification	✔ Completed
Day 5	June 05, 2025	Functional Use Case Preparation	✔ Completed
Day 6	June 06, 2025	Database Requirement Analysis	✔ Completed
Day 7	June 07, 2025	Entity-Relationship Design Maps	✔ Completed
Day 8	June 08, 2025	Relational Schema Implementation	⏳ In Progress

Day 1 — June 01, 2025 | Project Title Finalization

Project Overview

The Annual Report Portal (SIH1641) is an institutional enterprise platform designed to eliminate the highly fragmented and manual workflow of collecting, compiling, and formatting yearly performance parameters from various academic and administrative branches. This solution enables independent departments (e.g., Computer Science, Mechanical, Administration, Placements) to input data dynamically, while a centralized

core compilation engine aggregates, synthesizes, and builds highly customized institutional reports adhering to templates like NAAC, NIRF, or custom internal standards.

Justification for Title & Problem Code

- **'SIH1641':** Refers to the official Smart India Hackathon problem statement mapping requirements for streamlined institutional reporting.
- **'Annual Report Portal':** Identifies the core system target—compiling, formatting, validating, and generating the master annual institutional ledger.
- **'Integration & Customization':** Reflects the core engineering requirements—integrating distributed cross-departmental tabular schemas and customizing the layout templates dynamically.

Outcome

Title finalized and locked as "SIH1641: Annual Report Portal". Scope of multi-department configuration and customizable report templates confirmed with the project coordinator.

Day 2 — June 02, 2025 | Requirement Gathering

Functional Requirements

- **Dynamic Template Builder:** System Admins can create customizable form templates with dynamic fields (e.g., text, numbers, attachments) for data collection.
- **Department Submission Workbenches:** Heads of Departments (HODs) can fill, save drafts, edit, and formally sign off on their respective annual reports.
- **Central Consolidation Engine:** Ability to aggregate quantitative parameters (e.g., total research funds, placement counts, student intake) from all sub-units into a master dashboard.
- **Dynamic Formatting Options:** Custom formatting parameters enabling administrators to order sections, change font styles, and insert institutional seals or branding dynamically.
- **Approval Workflow:** Strict validation track tracking submissions from Department Staff → HOD Review → Institutional Committee Audit → Final Admin Release.
- **Multi-Format Exports:** Ability to download the compiled master report or isolated department sub-reports into cleanly paginated PDF documents and Excel sheets.

Non-Functional Requirements

- **Data Security & Integrity:** Enforced granular access lists, digital audit trails mapping every single value change back to a user ID, and transport layer security (SSL/TLS).
- **High Document Rendering Performance:** Asynchronous background parsing execution to compile 300+ page comprehensive institutional reports within 10 seconds without blocking system processes.

- **Extensible Configuration:** Data input fields must use flexible, generic structures to easily support shifting reporting templates year-over-year.
- **Operational Audit Logs:** Immutable historical tracking of template revisions, validation updates, and data sign-offs.

Outcome

Requirements Document locked, defining data flow paths from initial schema configuration down to final document layout compilation.

Day 3 — June 03, 2025 | Objective Definition

Primary Objectives

- To develop a secure, centralized web portal that automates data aggregation for annual institutional reporting across all independent branches.
- To implement an interactive layout engine allowing admins to customize, format, and re-order report elements dynamically.
- To eliminate systemic reporting errors through strict front-end validation rules and mandatory review workflows.
- To optimize reporting timeframes by providing a unified collaborative hub with automated deadline reminders.

Secondary Objectives

- To archive historical annual reports securely, enabling longitudinal trend analyses across departments.
- To support compliance initiatives by matching report builders with regulatory frameworks like NIRF and NAAC.

Scope Control Matrix

In Scope: Departmental onboarding, custom template generation, data entry modules, document approval routing, compilation engines, and PDF compilation layouts.

Out of Scope: External live database synchronization, automated web crawling for faculty citations, and continuous external document indexing systems.

Outcome

Established 4 primary technical targets and strict scope control metrics to prevent developmental scope creep.

Day 4 — June 04, 2025 | User & Module Identification

User Roles & Access Architecture

User Role	Functional Description	Access Privilege Level
System Admin	Initializes departments, creates metadata schemas, builds master templates, and generates final institutional reports.	Global Administrative Access
Department Head (HOD)	Reviews, corrects, and legally validates all report lines compiled by departmental staff before final lock.	Write/Validate Access (Own Department)
Department Staff	Enters quantitative logs, uploads project details, inputs publication records, and manages active entry logs.	Data Entry Access (Own Department)

System Module Mapping

Module Name	Core Capability Matrix	Primary User
Template Builder Module	Drag-and-drop form creation, dynamic attribute parameters, and variable schema structures.	System Admin
Data Entry Engine	Form entry interfaces, multi-category tabs, draft saving, and document upload handlers.	Department Staff
Workflow & Approval Engine	Submission validation tracking, review notes pipelines, rejection workflows, and formal status updates.	HOD, System Admin
Customization Layout Engine	Section ordering grids, cover configuration fields, and header/footer styling settings.	System Admin
Consolidation & Compilation	Calculates totals across departments and compiles final data into structured report templates.	System Admin

Outcome

Mapped 3 user access tiers against 5 core isolated system functional blocks.

Day 5 — June 05, 2025 | Use Case Diagram Preparation

Use Case Distribution Per Actor

👤 System Administrator Use Cases

- **UC-A01:** Configure New Institutional Department Profiles
- **UC-A02:** Build Dynamic Structural Data Collection Templates
- **UC-A03:** Monitor real-time Submission Compliance Dashboards
- **UC-A04:** Customize Master Compilation Page Orders & Document Layouts
- **UC-A05:** Execute Master Assembly Engine & Generate Consolidated Annual Report

👨‍💼 Department Head (HOD) Use Cases

- **UC-H01:** Access Departmental Overview Interface
- **UC-H02:** Review and Edit Input Metrics Compiled by Staff
- **UC-H03:** Return Sections to Staff with Detailed Review Feedback
- **UC-H04:** Approve and Lock Departmental Annual Records

✍️ Department Staff Use Cases

- **UC-S01:** Input Metric Fields into Active Report Sections
- **UC-S02:** Save Drafts & Upload Academic Proofs / PDF Documents
- **UC-S03:** Submit Completed Sections for Departmental Review

Outcome

Identified 12 core functional use cases, structuring development paths for the front-end user interfaces.

Day 6 — June 06, 2025 | Database Requirement Analysis

Entity Architecture & Relational Strategy

Entity Class	Key Schema Parameters	Structural Multiplicity & Constraints
PortalUsers	user_id, email, password_hash, account_role, department_id	Many Users match back to a single Department.
Departments	dept_id, dept_name, dept_code, head_user_id	One Department tracks multiple distinct submissions over the years.
ReportTemplates	template_id, execution_year, schema_definition_json, is_active	One Master Template orchestrates all specific sectional configurations.
ReportSections	section_id, template_id, title, section_order, field_blueprint_json	One Template groups multiple underlying Form Sections.
DeptSubmissions	submission_id, dept_id, template_id, tracking_status, locked_at	One Department registers precisely one submission row per active Year.
SectionData	data_id, submission_id, section_id, submitted_content_json, modified_by	Holds the actual data payloads corresponding to specific form sections.
AuditLogs	log_id, user_id, action_taken, target_record, execution_timestamp	Immutably tracks data entry actions across the portal lifecycle.

Outcome

Completed mapping of 7 persistent data entities using JSON fields to handle dynamic form layouts cleanly.

Day 7 — June 07, 2025 | ER Diagram Design

Relational Mappings

- **PortalUsers:** `user_id (PK)` | email | password_hash | account_role | department_id (FK)
- **Departments:** `dept_id (PK)` | dept_name | dept_code | head_user_id
- **ReportTemplates:** `template_id (PK)` | execution_year | schema_definition_json | is_active
- **ReportSections:** `section_id (PK)` | template_id (FK) | title | section_order | field_blueprint_json
- **DeptSubmissions:** `submission_id (PK)` | dept_id (FK) | template_id (FK) | tracking_status | locked_at

- **SectionData:** `data_id (PK)` | submission_id (FK) | section_id (FK) | submitted_content_json | modified_by (FK)
- **AuditLogs:** `log_id (PK)` | user_id (FK) | action_taken | execution_timestamp

Cardinality Definition

Source Node	Relationship Rule	Target Node	Multiplicity Type
Departments	contains operational accounts for	PortalUsers	One-to-Many (1:N)
ReportTemplates	defines structured configurations for	ReportSections	One-to-Many (1:N)
ReportTemplates	tracks dynamic compliance logs under	DeptSubmissions	One-to-Many (1:N)
Departments	compiles and routes data inside	DeptSubmissions	One-to-Many (1:N)
DeptSubmissions	holds specific data answers inside	SectionData	One-to-Many (1:N)
ReportSections	identifies structural layout for	SectionData	One-to-Many (1:N)

Outcome

Entity-Relationship integrity locked. Structural parameters handle both rigid relational tracking and flexible JSON fields cleanly.

Day 8 — June 08, 2025 | Database Schema Creation

The core DDL tables setup in PostgreSQL maps relational entities alongside JSONB data blocks to support dynamic form customization.

```
-- 1. Departments Master Data
CREATE TABLE Departments (
  dept_id SERIAL PRIMARY KEY,
  dept_name VARCHAR(150) NOT NULL,
  dept_code VARCHAR(10) UNIQUE NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Portal Users Registry
CREATE TABLE PortalUsers (
  user_id SERIAL PRIMARY KEY,
  email VARCHAR(120) UNIQUE NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
```

```
  account_role VARCHAR(30) NOT NULL CHECK (account_role IN ('Admin', 'HOD', 'Staff')),
  department_id INT REFERENCES Departments(dept_id) ON DELETE SET NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 3. Master Report Templates Core
CREATE TABLE ReportTemplates (
  template_id SERIAL PRIMARY KEY,
  execution_year INT UNIQUE NOT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 4. Template Sections Configurator
CREATE TABLE ReportSections (
  section_id SERIAL PRIMARY KEY,
  template_id INT NOT NULL REFERENCES ReportTemplates(template_id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  section_order INT NOT NULL,
  field_blueprint_json JSONB NOT NULL, -- Defines form input controls dynamically
  UNIQUE(template_id, section_order)
);

-- 5. Department Annual Status Tracking Tracker
CREATE TABLE DeptSubmissions (
  submission_id SERIAL PRIMARY KEY,
  dept_id INT NOT NULL REFERENCES Departments(dept_id) ON DELETE CASCADE,
  template_id INT NOT NULL REFERENCES ReportTemplates(template_id) ON DELETE CASCADE,
  tracking_status VARCHAR(30) DEFAULT 'Draft' CHECK (tracking_status IN ('Draft',
'Under_Review', 'Approved')),
  locked_at TIMESTAMP NULL,
  UNIQUE(dept_id, template_id)
);

-- 6. Section Submissions Structural Payload
CREATE TABLE SectionData (
  data_id SERIAL PRIMARY KEY,
  submission_id INT NOT NULL REFERENCES DeptSubmissions(submission_id) ON DELETE CASCADE,
  section_id INT NOT NULL REFERENCES ReportSections(section_id) ON DELETE CASCADE,
  submitted_content_json JSONB NOT NULL, -- Holds actual submitted dynamic data
  modified_by INT REFERENCES PortalUsers(user_id),
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 7. Platform Immutable Audit Log Trails
CREATE TABLE AuditLogs (
  log_id BIGSERIAL PRIMARY KEY,
```

```
  user_id INT REFERENCES PortalUsers(user_id) ON DELETE SET NULL,
  action_taken TEXT NOT NULL,
  execution_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Current Status

Database schema implementation is active and locked as of June 08, 2025. All 7 core tables have been structured with full relational constraints and are ready for validation testing.